CHORD SIMPLE IMPLEMENTATION

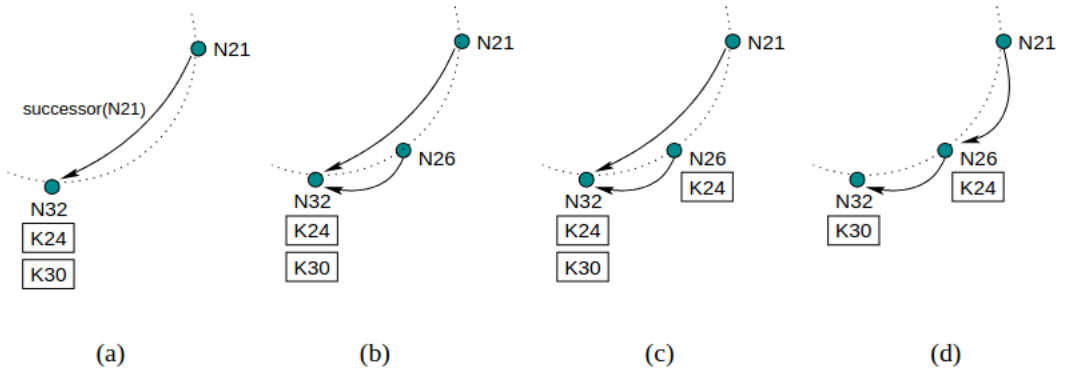
1. - Introducción

Se ha realizado una simple implementación de p2p basada en el algoritmo Chord tomando como referencia el paper *Chord: A Scalable Peer-to-peer Lookup Protocol for Internet Applications* [1]. Se ha implementado una versión simplificada de este algoritmo, permitiendo a los diferentes dispositivos unirse al anillo de Nodos y gestionar los recursos compartidos, pudiendo dar de alta, descargar y dar de baja recursos.
Aunque lo común en las implementaciones de protocolos p2p es utilizar sockets, nosotros lo hemos hecho con http, donde cada nodo actúa de cliente y servidor al mismo tiempo. Nos hemos tomado esta licencia ya que es un trabajo didáctico.

2.- Proceso

El entorno de desarrollo elegido ha sido Eclipse IDE, usando como lenguaje de programación Java en su versión 17, y usando las librerías de Jakarta 3.1.10 y commons codec 1.18. Las librerías de Jakarta nos permiten usar los servicios de REST para poder comunicar los distintos nodos entre sí, y la librería Commons Codec nos permite usar el algoritmo de Hashing SHA1. Por último, se ha utilizado un servidor de Tomcat 11 para ejecutar el proceso.

Primero se ha creado un anillo de tamaño 2^6 = 64 posibles ids, que varían entre 0 y 63. Luego, a través del método **join** hasheamos la IP de los nodos que se van añadiendo al anillo con módulo 64, comienza con la búsqueda de su sucesor a través del método **find_sucesor** y actualiza sus recursos quedándose con los recursos de su sucesor que tienen id menor que el id del nodo actual. A continuación se muestra un ejemplo de su funcionamiento donde, al añadirse el nuevo nodo con id=26 (figura b), el nodo 32 actualiza su predecesor y el nodo con id=26 se asigna los recursos con id menor o igual que 26.



Alejandro Martín Martín - 70832662E
Lur Perona Bzik - 53519833Y
Marta Muñoz del Fraile - 71970351Q

Cada cierto tiempo los nodos llaman a la función de estabilizar y notificar, que actualiza los sucesores y predecesores de todos los nodos. Esto asegura que los enlaces entre nodos sean correctos.

3.- Resultado

La implementación realizada funciona de forma correcta, pero hay que tener en cuenta las siguientes consideraciones:

- El diseño realizado es completamente escalable.
- Pueden compartirse todo tipo de ficheros, pues a la hora de compartirse se transforman en base64.
- Es necesario introducir la IP del Nodo de entrada (es decir, la ip de un nodo que ya esté en el anillo) directamente en el código de este. Esto podría solucionarse si al ejecutar el programa pudiéramos pasar esa IP como argumento.
- Debido a que cada nodo sólo conoce a su sucesor y a su predecesor, si se cae 1 nodo el anillo quedaría roto. Esto lo hemos solucionado de forma en que cuando un nodo descubre que su sucesor se ha caído, vuelve a ejecutar la función Join contra el nodo de entrada, por lo que debemos de suponer que ese nodo NUNCA puede caerse. Esto podría solucionarse haciendo que cada nodo conozca quienes son sus r sucesores. Con esto el anillo se rompería si se caen r nodos consecutivos, cosa que es muy improbable para valores modestos de r.
- El algoritmo de búsqueda de sucesores y de archivos (findSucesor) tiene una complejidad de O(n), lo cual no es muy adecuado en anillos muy grandes. En el paper se propone una solución llamada *finger_tables,* pero dada su complejidad hemos decidido no implementarlo.

4.- Bibliografía

[1] Stoica, I., Morris, R., Liben-Nowell, D., Karger, Kaashoek, M., Dabek, F., & Balakrishnan, H. (2003). Chord: a scalable peer-to-peer lookup protocol for internet applications. *IEEE/ACM Transactions On Networking*, *11*(1), 17-32. https://doi.org/10.1109/tnet.2002.808407

Alejandro Martín Martín - 70832662E
Lur Perona Bzik - 53519833Y
Marta Muñoz del Fraile - 71970351Q